

Advanced Cryptography

Elisabeth Oswald

Revision Lecture 1 — Security Notions, Proofs, Secret Sharing for MPC

ROADMAP

Security Notions

- Notions expressed as Games

- Showing insecurity with games

- Relationships between LR and RR notions

Secret Sharing

- Basics

- Proving security

MPC

PERFECT SECRECY (FOR ONE-TIME ENCRYPTION)

Symmetric encryption: given some well define message space, key space and ciphertext space, we have three algorithms that define the scheme. Key Gen to select a key from the key space, Enc for encryption (taking in a message and a key), and Dec for decryption (taking in a ciphertext and a key).

bf One-time encryption: we use a secret key only once for a message.

(Perfect) secrecy: Given a symmetric encryption scheme $\mathcal{E}$, we wish to capture the intuition that no adversary learns anything further about a message upon seeing the ciphertext of the message.

“Further??”: than what they know because they know the definition of the message space.

“Perfect”?: if this property holds even for computationally unlimited adversaries.

PERFECT SECRECY EXPRESSED IN TWO WAYS

An encryption scheme Gen, Enc, Dec with message space $\mathcal{M}$ is **perfectly secret** if for every probability distribution over $\mathcal{M}$, every message $m \in \mathcal{M}$, and every ciphertext $c \in \mathcal{C}$ for which $\Pr[C = c] > 0$:

$$\Pr[M = m | C = c] = \Pr[M = m].$$

The picture on the right shows a game that can be proven to be equivalent to the mathematical definition.

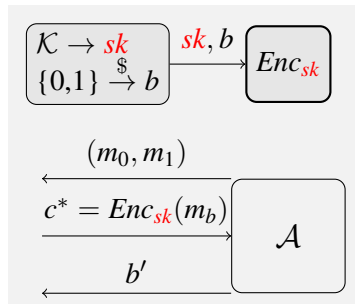


Figure: Exp ind-pas. Win if $b' = b$.

Modern notions “rewrite” the game once more to more clearly express the idea of “two worlds” that cannot be distinguished: see next slide.

ONE-TIME ENCRYPTION: OTP

One-time pad encryption

Message space $\mathcal{M}$, key space $\mathcal{K}$, and ciphertext space $\mathcal{C}$ are all the set $\{0, 1\}^l$, aka l -bit strings.

Key Gen: pick a key uniformly, at random (aka all keys are chosen with equal probability).

Enc: $c = m \oplus k$ (exclusive-or, aka, addition modulo 2 of two l -bit strings).

Dec: $m = c \oplus k$.

Example: $l = 1$ and $m = 0$. With key $k = 0$ we have $c = 1$, with $k = 1$ we have $c = 0$.

For a fixed m the ciphertext space is a permutation of the key space: each m sends a key k to exactly one c . This one-to-one mapping implies that if keys are picked with equal probability, then also ciphertexts are picked with equal probability.

INDISTINGUISHABILITY OF CIPHERTEXTS: ONE-TIME ENCRYPTION

Game is now specialised to the one-time pad (OTP). The game master flips a bit b **to select the left or right world** $\{L, R\}$.

The adversary produces two messages of equal length; they the the encryption of one of them.

Their task is to decide though whether c comes from the left or the right world.

If we can prove that an adversary cannot distinguish between the worlds, then the encryption scheme is (perfectly) secret.

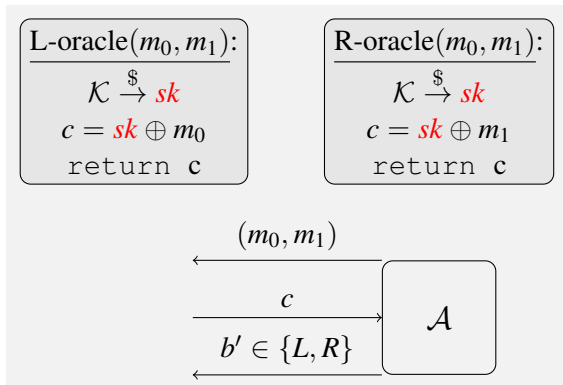


Figure: Exp LR-Pas. Win if $b' = b$.

INDISTINGUISHABILITY OF CIPHERTEXTS: ONE-TIME ENCRYPTION

How can we show indistinguishability ? By proving that the distribution of ciphertexts c of the left world is exactly the same as the distribution of ciphertexts produced by the right world.

The key sk is chosen from a uniform distribution, the message m_0 is a fixed quantity in the left world. The Boolean exclusive-or function $m \oplus sk$ acts like a permutation=bijection (on the key space): the probability distribution of $c = m_{0,1} \oplus sk$ is the same as the probability distribution of k .

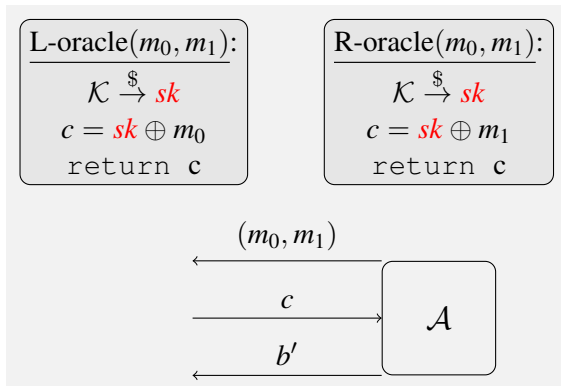


Figure: Exp LR-Pas. Win if $b' = b$.

The **distribution of outputs** in the left oracle is the same as in the right oracle for any choice of inputs.

REAL VS RANDOM SECURITY: OTP

The next step is the mental model behind simulation based security.

Distinguish between result of an oracle that resembles the “real world” vs an oracle that resembles the “ideal” world.

In the “ideal world” there is no key, and return values are random values (hence “random” world.).

For the one-time pad this holds: the distribution of the real world outputs is uniform of size $|\mathcal{K}| = |\mathcal{C}|$, which is the same as in the random world.

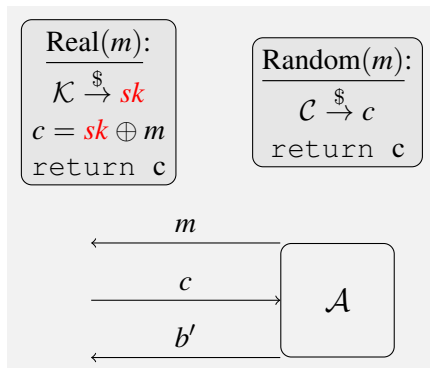


Figure: Exp RR-Pas. Win if $b' = c$.

ROADMAP

Security Notions

Notions expressed as Games

Showing insecurity with games

Relationships between LR and RR notions

Secret Sharing

Basics

Proving security

MPC

EXAMPLE OF INSECURE ENCRYPTION: LR

We define the Multi-Time Pad (MTP) as the OTP but sample only a single key bit and replicate this $l - 1$ times if we get an l -bit message/ciphertext.

Example: $m = 0000$, with $k = 1$, this gives $c = 1111$.

Attack: choose $m_0 = 000$ and $m_1 = 001$. The left world can only produce 000 or 111 as ciphertexts. The right world can only produce 001 or 110. So the pattern of the ciphertext tells us which world was used.

The **distribution of outputs** in the left oracle is not the same as in the right oracle.

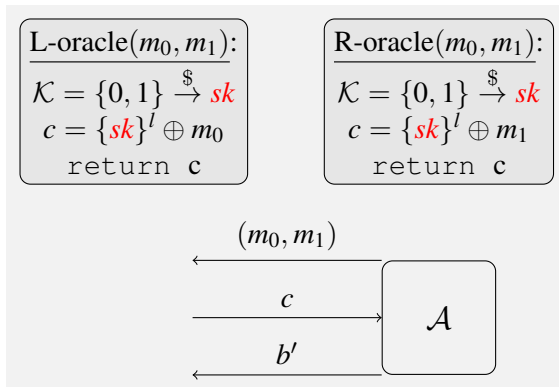


Figure: Exp LR-Pas. Win if $b' = b$.

EXAMPLE OF INSECURE ENCRYPTION: RR

We use the same MTP encryption scheme.

We choose a message $m = 0000$.

The Real world can only ever produce either 0000 or 1111 as ciphertext.

The Random world will produce any length 4 bit string.

Clearly the output distributions of the Real and Random worlds are very different.

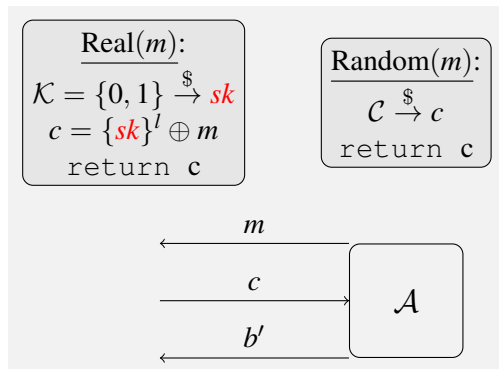


Figure: Exp RR-Pas. Win if $b' = b$.

ROADMAP

Security Notions

Notions expressed as Games

Showing insecurity with games

Relationships between LR and RR notions

Secret Sharing

Basics

Proving security

MPC

RR SECURITY IMPLIES LR SECURITY (BUT NOT VICE VERSA)

Claim: if an encryption scheme is real-random secure (i.e. the outputs from the real oracle cannot be distinguished from outputs of the random oracle), then, it is also left-right secure.

(This is exercise 5 from the formative exercises on lab sheet 1.)

RR SECURITY IMPLIES LR SECURITY

Assumption: we have an encryption scheme, that provably has RR security.

Argument: if we can “rewrite” the e.g. left oracle into the right oracle, without changing the functionality, then this implies that the left and right oracle produce the same output distributions.

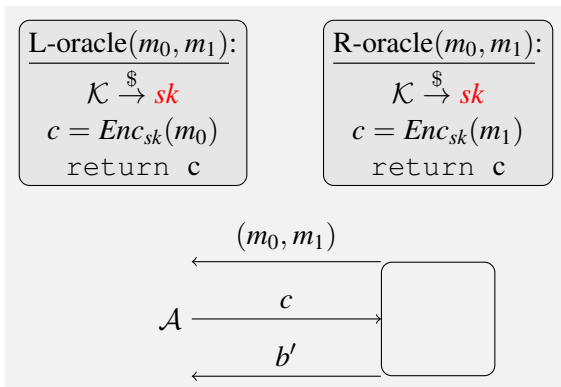


Figure: Exp LR-Pas. Win if $b' = b$.

RR SECURITY IMPLIES LR SECURITY, CONT.

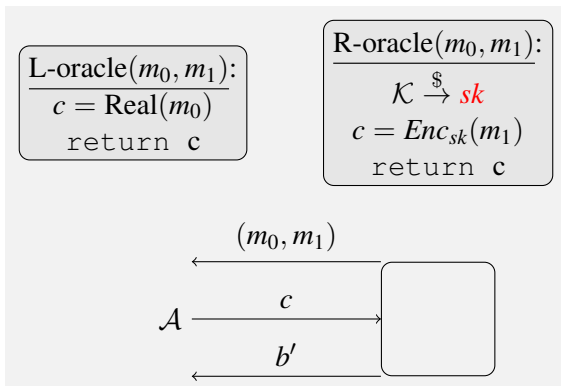


Figure: Exp LR-Pas. Win if $b' = b$.

Next observe that we know that the encryption scheme has RR security: so the output distribution of the Real oracle is indistinguishable from the output distribution of the Random oracle.

RR SECURITY IMPLIES LR SECURITY, CONT.

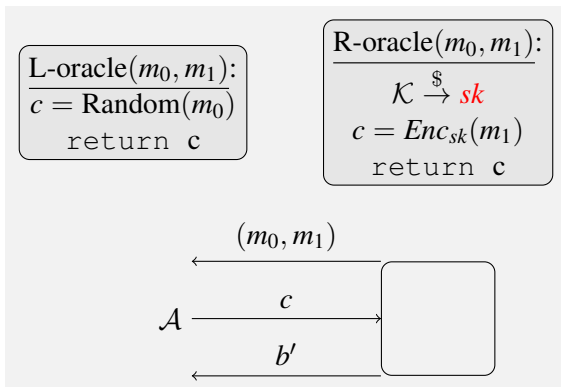


Figure: Exp LR-Pas. Win if $b' = b$.

Now observe that in the code of Random oracle, the input m_0 is actually never used. So we change m_0 to m_1 without changing the functionality of the Random oracle.

RR SECURITY IMPLIES LR SECURITY, CONT.

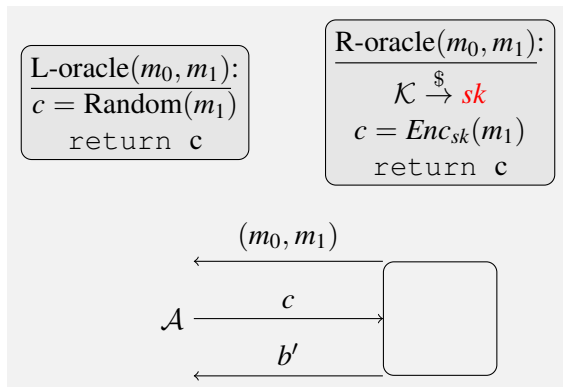


Figure: Exp LR-Pas. Win if $b' = b$.

Remember that the output distributions of Real and Random oracles are indistinguishable.
So now we are going to swap the Random for the Real world.

RR SECURITY IMPLIES LR SECURITY, CONT.

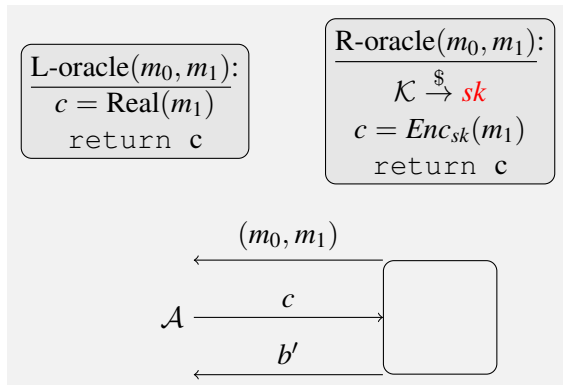


Figure: Exp LR-Pas. Win if $b' = b$.

Finally, we are going to expand out the code for the Real world.

RR SECURITY IMPLIES LR SECURITY, CONT.

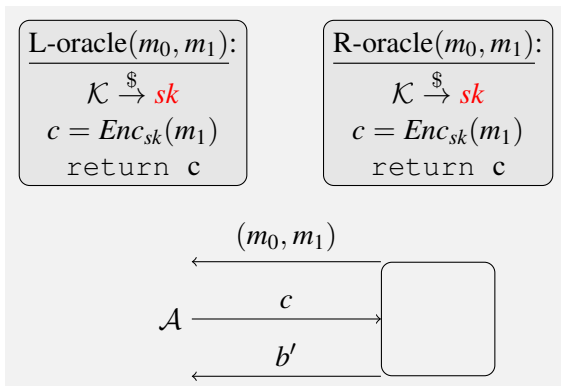


Figure: Exp LR-Pas. Win if $b' = b$.

Yeah. The code in our left world is now exactly the same code as in the right world. Which does imply that the output distribution of the left world must match the output distribution of the right world. Which is what we wanted to show.

LR SECURITY DOES NOT IMPLY RR SECURITY

Define a scheme where the message space $\mathcal{M}$ and key space $\mathcal{K}$ are the set $\{0, 1\}^l$, and ciphertext space $\mathcal{C}$ is $\{0, 1\}^{l+2}$

Key Gen: pick a key uniformly, at random.

Enc: $c_1 = m \oplus k, c = c_1 || 11$. (All ciphertexts end on the string 11.)

Dec: $c_1 = c - \{11\}, m = c_1 \oplus k$.

Example: $l = 1$ and $m = 0$. With key $k = 0$ we have $c = 111$, with $k = 1$ we have $c = 011$.

This scheme has LR security: it is the OTP with two trailing bits (the same in the left and the right world).

This scheme cannot have RR security with uniform ciphertexts; we could redefine the ciphertext space though to fix the last two bits.

ROADMAP

Security Notions

Notions expressed as Games

Showing insecurity with games

Relationships between LR and RR notions

Secret Sharing

Basics

Proving security

MPC

Secret Sharing Scheme

Given a message space $\mathcal{M}$, a threshold value $t \in \mathbb{N}$, and a number of shares $n \in \mathbb{N}$, with $t < n$.

A (t, n) **threshold secret sharing scheme** Σ consists of two algorithms:

Share: takes a message $m \in \mathcal{M}$ as input, and outputs an n -tuple of randomly selected shares $\mathbf{s} = (s_1, \dots, s_n)$.

Reconstruct: takes a subset of t or more shares as input, and produces a message m' as output.

We are only interested in **correct and secure** schemes.

Perfectly secure secret sharing

Let Σ be a (t, n) threshold secret sharing scheme with message space $\mathcal{M}$. We say that Σ is **perfectly secure** if for all choices of messages $m_0, m_1 \in \mathcal{M}$, no unauthorised subset of users $\mathcal{U}$ can distinguish between the shares of m_0 and m_1 .

Notice that the security notion here relates to LR security (rather than RR security).

Boolean secret sharing: often proof arguments use RR security because Boolean secret sharing is essentially OTP encryption.

Shamir secret sharing is based on polynomials and we show that it is also information theoretically secure. So proof arguments are often RR style.

EXAMPLE: DISTRUSTING ORGANISATIONS

Lab Sheet 1, Exercise 6:

Consider the situation where MI5 and MI6 want to share the secret combination to a vault.

They want this to be such that five employees of MI5 and six employees of MI6 together can open the vault.

So only if eleven employees from both organisations come together (five from MI5 and six from MI6), should they be able to reveal the secret combination.

How would you deal with this situation? Who would be creating the shares, and how would share distribution perhaps work in practice?

EXAMPLE: DISTRUSTING ORGANISATIONS, CONT.

The description makes clear that exactly 5 employees from MI5 and exactly 6 employees from MI6 must come together to reveal the secret combination, which I call s . The length of l of s is not specified by the example, neither from which alphabet/space s comes from. It is also not clear if a threshold scheme is required or not.

But it is clear that s needs to be distributed among the two organisations, so we must have $s = s_1 + s_2$, whereby s_1 will be secret shared among employees of *MI5*, and s_2 will be secret shared among employees of *MI6*. The addition has yet to be defined.

One legitimate solution here is to say that s is a value that can be interpreted as a bit string, and therefore we share out s via $s = s_1 \oplus s_2$ using Boolean secret sharing.

Perhaps MI5 prefers Boolean secret sharing, and they have identified exactly 5 trusted members of staff. Perhaps MI6 prefers Shamir's secret sharing and they have identified 10 trusted members of staff.

EXAMPLE: DISTRUSTING ORGANISATIONS, CONT.

Perhaps everybody prefers that nobody should ever know the secret combination s , so they have commissioned a piece of hardware, that will do all the necessary crypto operations and just spit out the resp. secret shares upon identification of the involved individuals.

Then this machine is set up to:

1. Randomly sample a 10 character code and translate this into a binary string, resulting in e.g. an 80-bit value s .
2. Boolean secret share s , giving s_1 and s_2 .
3. Destroy s .
4. Boolean secret share s_1 , giving $s_{1,i}$ (and $i = 1, 2, \dots, 5$).
5. Destroy s_1 .
6. Shamir secret share s_2 , with $n = 10$ and $t = 6$, giving $s_{2,j}$ (and $j = 1, 2, \dots, 10$).
7. Destroy s_2 .
8. Upon identification of users, give out the resp. shares.

SHAMIR'S SCHEME

A secret $m \in \mathbb{Z}_p$ is to be shared with a threshold of t .

We choose a secret degree $(t - 1)$ polynomial f , that satisfies $f(0) \equiv m \pmod{p}$.

This is another way of saying that the coefficient a_0 in our notation is exactly $m \pmod{p}$.

All other coefficients are chosen randomly from $\mathbb{Z}_p$.

The i -th user receives the point $(i, f(i) \pmod{p})$ (and $i > 0$).

All operations must always be understood to happen modulo p . All parameters, e.g. n, t, m must be chosen to be smaller than p .

SHAMIR'S SCHEME: SHARE AND RECONSTRUCT

```
1  def Stn.Share(m):  
2       $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{Z}_p$   
3       $f(x) = m + \sum_{j=1}^{t-1} a_j x^j$   
4      for i=1 to n:  
5           $s_i = (i, f(i))$   
6      return  $\mathbf{s} = (s_1, \dots, s_n)$ 
```

Listing: Shamir (t, n) secret sharing: Share function

```
1  def Stn.Reconstruct( $\{s_i | i \in \mathcal{U}\}$ ):  
2      If  $|\mathcal{U}| \geq t$ :  
3          Derive unique degree  $t-1$  polynomial  $f(x)$   
4          return  $f(0)$   
5      else  
6          return  $\perp$ 
```

Listing: Shamir (t, n) secret sharing: Reconstruct function

RECONSTRUCT

Given t pairs $(x_i, f(x_i))$, construct the so-called Lagrange polynomials

$$\delta_i(x) = \frac{\prod_{j \neq i} (x - x_j)}{\prod_{j \neq i} (x_i - x_j)} \pmod{p}.$$

These polynomials have the property that given a pair $(x_i, f(x_i))$, the polynomial $\delta_i(x_i)$ evaluates to 1 and it evaluates to 0 otherwise.

Therefore the polynomial $f(x)$ is given as:

$$f(x) = \sum_{i=1}^{d+1} f(x_i) \delta_i(x) \pmod{p}.$$

Note that $f(x_i)$ is NOT a function here but a value in $\mathbb{Z}_p$.

And the secret can be retrieve by evaluating $f(0)$.

RECONSTRUCT, CONT.

Notice that $f(0)$ really is:

$$f(0) = \sum_{i=1}^{d+1} f(x_i) \delta_i(0) \pmod{p}.$$

Thus for any set of t legitimate users, there is a set of $\delta_i(0)$. These values can be calculated by anyone, as there is no secret ingredient anywhere:

$$\delta_i(0) = \frac{\prod_{j \neq i} (0 - x_j)}{\prod_{j \neq i} (x_i - x_j)} \pmod{p}.$$

Therefore the polynomial $f(0)$ is a linear combination of these $\delta_i(0)$. Only the $f(x_i)$ in this linear combination are secret — they are the shares of the resp. parties.

For any set of legitimate users, we can therefor compute/publish/assume known a so-called **public recombination vector**, which is the corresponding set of $\delta_i(0)$.

RECONSTRUCT, EXAMPLE

Example 7, from Lab Sheet 1.

We have parties $P_1, \dots, P_5$ who shared a secret s via a $(2, 5)$ Shamir secret sharing scheme over $\mathbb{Z}_{101}$. Compute the public recombination vector for the subset $\{P_1, P_2\}$ and for the subset $\{P_3, P_5\}$. Does the publication of these vectors help P_4 ?

The example refers to a $(2, 5)$ secret sharing scheme, so two parties suffice for reconstruction.

The example gives two sets of two parties and asks for the public recombination vectors for both of these sets.

For the subset $\{P_1, P_2\}$ the recombination vector has coefficients 2, 100, for the subset $\{P_3, P_5\}$ the recombination vector has coefficients 52, 48.

There is no help for P_4 here because P_4 could compute these vectors anyway.

RECONSTRUCT, EXAMPLE

Example 9, Lab Sheet 1.

You set up a $(2, 30)$ Shamir secret sharing scheme over $\mathbb{Z}_{101}$. You gave out shares to David $(1, 13)$ and Tom $(3, 12)$, and also to Mark who is the second user in this system. Sadly he lost his share. Can David and Tom help him recover it?

As the “second user”, Mark has x -coordinate 2. David and Tom can come together to reconstruct $f(x)$ by using Lagrange interpolation, and then they compute $f(2)$ and give this to Mark. In this example, Mark’s share is 63.

Obviously, as part of the process, David and Tom can also reconstruct the secret.

ROADMAP

Security Notions

Notions expressed as Games

Showing insecurity with games

Relationships between LR and RR notions

Secret Sharing

Basics

Proving security

MPC

REAL AND IDEAL=RANDOM ORACLES

```
1  def Share(m, t,  $\mathcal{U} \subset \{1, \dots, p\}$ ) :  
2      if  $|\mathcal{U}| < t$ :  
3          return  $\perp$   
4       $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{Z}_p$   
5       $f(x) = m + \sum_{j=1}^{t-1} a_j x^j$   
6      for i=1 to n:  
7           $s_i = (i, f(i) \pmod{p})$   
8      return  $\{s_i | i \in \mathcal{U}\}$ 
```

Listing: Real world

```
1  def Share(m, t,  $\mathcal{U} \subset \{1, \dots, p\}$ ) :  
2      if  $|\mathcal{U}| < t$ :  
3          return  $\perp$   
4      for  $i \in \mathcal{U}$ :  
5           $y_i \xleftarrow{\$} \mathbb{Z}_p$   
6           $s_i = (i, y_i)$   
7      return  $\{s_i | i \in \mathcal{U}\}$ 
```

Listing: Ideal world

OUTPUT DISTRIBUTIONS

Real world: the probability of output i, y_i depends on the random choice in line 4, and the “restriction” that the inputs put on the sampled polynomial:

- ▶ There are $t - 1$ random coefficients, i.e. p^{t-1} polynomials overall.
- ▶ These polynomials must through the supplied x_i values, and therefore certain $f(x_i)$ values. There are $|\mathcal{U}|$ such values, this means that there are only $p^{t-1-|\mathcal{U}|}$ polynomials with this property.
- ▶ Thus the probability to produce a polynomial that goes through $x_i, f(x_i)$ is $p^{-|\mathcal{U}|}$.

Ideal world: we pick $|\mathcal{U}|$ values for $f(x_i)$, i.e. the probability for picking such $|\mathcal{U}|$ values is $p^{-|\mathcal{U}|}$.

Ergo: the output distributions of both worlds are exactly the same.

LR security follows immediately from the fact that we have just shown RR security.

Multiple parties supply input(s) X_i and wish to jointly evaluate a function $f(X_1, X_2, \dots)$.

All parties participate in the computation of $f()$, but perhaps not all parties should be required in revealing the actual result.

It could also be that external parties do the computations.

Therefore we look at threshold multiparty computation, where each party provides (Shamir) secret shares for their inputs.

Consequently: we must have all inputs as values modulo p , and also the function must be expressed/evaluated modulo p . Obviously p must be large enough.

EXAMPLE: ADDITION WITH THREE PARTIES

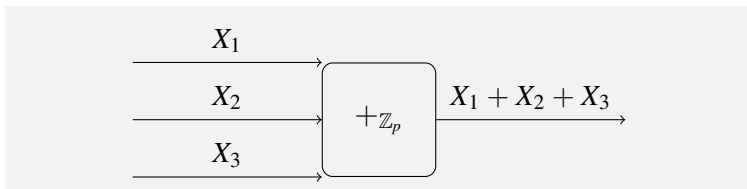


Figure: Modular addition with inputs from three parties: situation without secret sharing.

Suppose three parties wish to aggregate their inputs using a third untrusted party.

So in this example, the parties are not directly computing the sum, but another party is doing this for them.

EXAMPLE: MPC ADDITION WITH A (2, 2) SECRET SHARING SCHEME

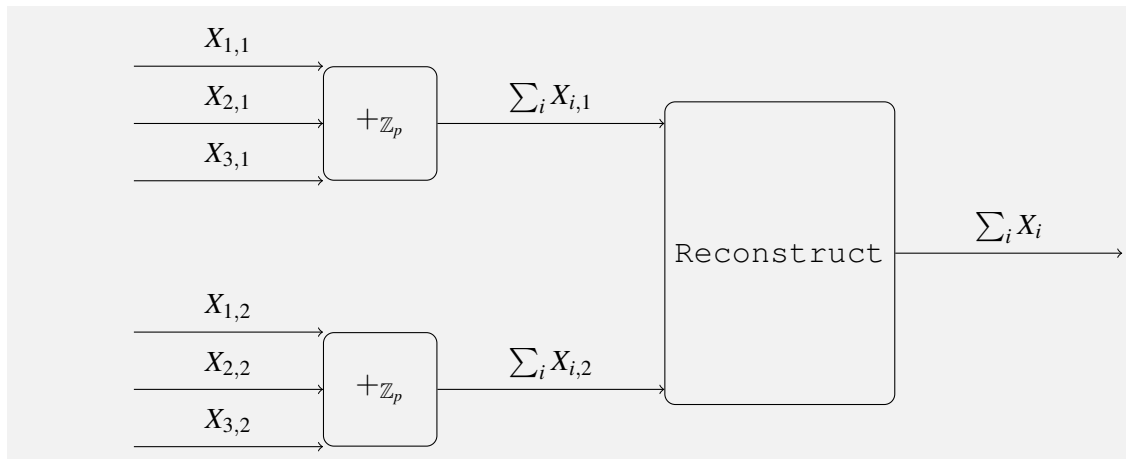


Figure: Modular addition with inputs from three parties using two independent aggregators.

BGW PROTOCOL

Implements a Shamir secret sharing based MPC solution.

Functions are expressed as arithmetic circuits modulo p . All parties who compute have the same circuit.

There are three basic operations (modulo p) that each function can be expressed in: addition, multiplication with a constant, multiplication.

Two of them do not require special considerations: these are the addition and the multiplication with a constant.

The non-linear operation is the multiplication: the resulting polynomial is of too high degree. This requires a trick called degree reduction, which requires that all computing parties must exchange messages.

DEGREE REDUCTION

Multiplication: if a is secret shared via $f(x)$ and b is secret shared via $g(x)$, then the product is $h(x) = f(x) \cdot g(x)$, and $h(0) = c = a \cdot b$ with degree $2t$. Assume we have enough parties, so $n \geq 2t$.

Each party P_i has one share of h , say $h(i)$.

We know that $c = h(0) = \sum_i \delta_i(0)h(i)$: the secret is represented as a linear combination of shares $h(i)$ using the public recombination vector.

The idea behind the degree reduction is to just produce another linear combination: one that corresponds to lower degree polynomials.

We do this by sampling new degree t polynomials that have $h(i)$ as their zero-th coefficient.

DEGREE REDUCTION, CONT.

I.e. to “re-share” the $h(i)$ values, we consider them as secrets of new, degree t polynomials $\delta'_i(x)$. So $\delta'_i(0) = h(i)$.

Therefore we know that $H(0) = \sum_i \delta_i(0)\delta'_i(0) = \sum_i \delta_i(0)h(i) = h(0) = c$.

So each party re-shares their $h(i)$ by generating a $\delta'_i(x)$ with $\delta'_i(0) = h(i)$, and sends the resp. share to the other parties.

Any party P_j has shares $\delta'_i(j)$, and can therefore compute their new share of the H as $H(j) = \sum_i \delta_i(0)\delta'_i(j)$.

MPC — BGW EXAMPLE 11, LAB SHEET 1

We consider an example with parameters $(2, 3)$ (i.e. three parties in total, two are required to reconstruct). The inputs of the parties are thus shared with degree $d = 1$ polynomials. All computations happen in $\mathbb{Z}_{13}$.

Value	P_1	P_2	P_3
a	5	8	11
b	7	9	11
$a \cdot b$	c_1	c_2	c_3

What are their shares c_i for the product initially? Verify that they hold a correct secret sharing of the product 10 via the public recombination vector.

MPC — BGW EXAMPLE 11, LAB SHEET 1, CONT.

You get the c_i by multiplying the shares that each party has. This is because $h(x) = f(x) \cdot g(x) = f \cdot g(x)$. You should find that $c_1 = 9$, $c_2 = 7$, and $c_3 = 4$.

To verify that this is correct you have two options: reconstruct the polynomials $f(x)$ and $g(x)$, multiply them, and then check with the resp. coordinates; or, which is what I instruct, compute the public recombination vector. To reconstruct, all parties are required: because the degree of $h(x)$ is two, so we need three parties.

$$\delta_1(0) = \frac{(0-2)(0-3)}{(1-2)(1-3)} = 3$$

$$\delta_2(0) = \frac{(0-1)(0-3)}{(2-1)(2-3)} = -3 \equiv 10$$

$$\delta_3(0) = \frac{(0-1)(0-2)}{(3-1)(3-2)} = 1.$$

$$\{P_1, P_2, P_3\}: c = 3c_1 + 10c_2 + c_3 = 3 \cdot 9 + 10 \cdot 7 + 1 \cdot 4 = 10.$$

MPC — BGW EXAMPLE 11, LAB SHEET 1, CONT.

Then perform a degree reduction by assuming that each party uses the following secret polynomials:

$$P_1 : c_1 + 2x$$

$$P_2 : c_2 + 5x$$

$$P_3 : c_3 + 1x.$$

Work out the shares that all P_i produce and make clear which share is for which party. Then assume that the shares are sent out. Write down which shares the parties hold after they receive shares from the other parties.

MPC — BGW EXAMPLE 11, LAB SHEET 1, CONT.

$$P_1 : 9 + 2x$$

$$P_2 : 7 + 5x$$

$$P_3 : 4 + 1x.$$

Therefore the party P_1 produces shares 11, 0, 2, the party P_2 produces the shares 12, 4, 9 and the party P_3 produces the shares 5, 6, 7. Party P_1 sends the shares 0, 2 to the parties 4, 9, etc. So after resharing, the parties hold shares for $H(0)$:

Value	P_1	P_2	P_3
$a = 2$	5	8	11
$b = 5$	7	9	11
$h(0) = 10$	9	7	4
$H(0) = 10$	2	7	12

MPC — BGW EXAMPLE 11, LAB SHEET 1, CONT.

With the new shares for the product, show, with the public recombination vector, that they give the correct value of the product.

Finally, consider what happens if P_2 is taken over by a malicious adversary who changes the value of their share to 9 in the last step.

$$\{P_1, P_2, P_3\}: c = 3c_1 + 10c_2 + c_3 = 3 \cdot 2 + 10 \cdot 7 + 1 \cdot 12 \equiv 6 + 5 + 12 \equiv 10.$$

If in that final step, P_2 changed their share from 7 to 9, then they would find that the product is 4. They have no means to notice that this is in fact not the correct product. Thus we can see that Shamir's secret sharing scheme might well offer perfect input privacy, but it offers no robustness.

MPC — ONLY A BILLIONAIRE'S PROBLEM, LAB SHEET 1

Compute a sum and compute the largest input.

Question 1: what solutions for MPC are out there? Requires you to do research with the aim of giving you a bit of a “bigger picture” view. IMHO the nicest, curated list for MPC resources is this: <https://github.com/rdragos/awesome-mpc>.

Question 2: explain how to do a sum over integers. BGW addition with a large enough modulus will do just that.

Question 3: do a maximum with Shamir's secret sharing. This means that a BGW like approach is hinted at. But you could try and find something else too. The issue is that a maximum is a pain to compute when working modulo p . Some form of tree based subtracting circuit needs to be done. So for a maximum, a Boolean circuit would be better suited. Ergo, in many “real world” MPC schemes/libraries, one actually has functionality to switch between Boolean secret sharing, Garbling (which we did not get around to), and BGW-like computations.